



**SISTEMAS OPERATIVOS - LABORATORIO**  
**18 de Diciembre de 2025, Turno 2**

Nombre \_\_\_\_\_ DNI \_\_\_\_\_  
Apellidos \_\_\_\_\_ Grupo \_\_\_\_\_

**INSTRUCCIONES**

- Se debe entregar una carpeta individual con el código de cada apartado (1A, 1B, etc.), para así garantizar la evaluación independiente de cada uno.
- Si el código no compila o su ejecución produce un error grave la puntuación de ese apartado será 0.

**Cuestión 1. (5 puntos)** Implementar un programa que procese en paralelo varios ficheros de texto que contienen los elementos de una matriz, y realice una suma o producto de los elementos de cada fila de las matrices. Con el enunciado se proporciona una plantilla para la solución. La forma de uso del programa será:

`./matrix_comp -o "+"|"*" fichero1 [fichero2 ...]`

El programa recibe con la opción -o una operación a realizar ("+" ó "\*", deben pasarse entrecomilladas para evitar la expansión del shell) sobre los elementos de las filas de las matrices; y una lista de ficheros a procesar. El vector columna resultante de operar sobre cada matriz se guardará en un fichero de salida. Para aprovechar los *cores* del sistema, cada matriz será procesada por un proceso diferente, mientras el principal espera a que los demás terminen, informando al usuario de su estado de finalización. Los pasos a seguir y la puntuación asociada a cada uno son:

- (1 punto)** Crear un programa que procese con `getopt` el flag -o, cuyo argumento debe ser la cadena de caracteres "\*" ó "+". Si el usuario pasa cualquier otro valor, el programa debe mostrar el mensaje de uso y terminar. Después el programa debe recorrer la lista de ficheros que se le pasan por la línea de comandos, y para cada uno ellos:
  - Generar el nombre del fichero de salida, añadiendo ".out" como extensión al nombre del fichero.
  - Invocar la función `matrix_compute` (ver plantilla) pasando el nombre del fichero original, el nombre del fichero de salida y la operación a realizar.
- (1,5 puntos)** Modificar el bucle del apartado a) que recorre los ficheros pasados como argumento para que por cada fichero se cree un nuevo proceso que invoque la función `matrix_compute`, sin esperar a que terminen los procesos hasta que se hayan lanzado todos.
- (1,5 puntos)** La función `matrix_compute` cambiará el programa en ejecución (con `exec`) por `"/bin/bash -c cmd"`, donde `cmd` será el comando siguiente: `sed 's/ /<op>/g' <in_file> | bc > <out_file>`. Nótese que `<op>` será el carácter '+' en caso de suma o el carácter '\*' en caso de multiplicación. Asimismo, `<in_file>` y `<out_file>` deben ser sustituidos por las cadenas que se pasan como argumento a la función. **Recomendación:** utilizar `snprintf` con la cadena de formato incluida en la plantilla.
- (1 punto)** Tras haber lanzado todos los procesos, el programa principal debe esperar a que terminen. Cada vez que termine un proceso mostrará por la salida estándar un mensaje indicando el pid del proceso que ha finalizado, el fichero que procesó dicho proceso, su estado de finalización (si ha terminado por `exit` o murió por la recepción de una señal) y el valor de retorno al sistema (o señal que lo terminó, según el caso).

Ejemplo:

|                                                                                                                                                                                                                                                          |                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <pre>\$ cat 3x3.txt 1 2 3 4 5 6 7 8 9 \$ cat 5x5.txt 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4 5 5 5 5 5 \$ ./matrix_comp -o + 3x3.txt 5x5.txt [404677] operating on 3x3.txt exited with status 0 [404678] operating on 5x5.txt exited with status 0</pre> | <pre>\$ cat 3x3.txt.out 6 15 24 \$ cat 5x5.txt.out 5 10 15 20 25</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|

**Cuestión 2. (5 puntos)** Realiza una modificación de la práctica 6, en la que cada salvaje deba, al finalizar su comida, pagar en un cajero las raciones que haya consumido. El cajero será implementado en un nuevo programa, que se sincronizará con los procesos salvajes utilizando tres semáforos con nombre y dos variables compartidas, *last\_servings* y *last\_payment*; atendiendo a las siguientes reglas:

- Los salvajes deben pagar de uno en uno, evitando posibles carreras.
- Cuando un salvaje vaya a pagar debe poner en *last\_servings* el número total de raciones que ha consumido y en *last\_payment* el dinero correspondiente (cada ración costará *MENUPRICE*, con valor 10). Luego deberá avisar al cajero de que el dinero está depositado y quedarse esperando a que el cajero lo retire.
- El cajero estará siempre esperando hasta que un salvaje le avise de que ha depositado el dinero, entonces lo retirará y actualizará la cuenta total de raciones (*total\_servings*) y de ingresos (*total\_incomes*), que son variables globales del proceso cajero. Luego avisará al salvaje para que éste pueda continuar (terminar), dando paso entonces al siguiente salvaje.

**Sugerencia:** puedes usar el mismo patrón que con el cocinero, utilizando uno de los semáforos como cerrojo (inicializado a 1) y los otros dos como colas (inicializados a 0).

A continuación te damos una guía de pasos a seguir, con la valoración de cada uno de ellos:

- (1 punto)** Crea un nuevo programa (*cashier.c*), que hará de cajero, tomando como referencia el código del cocinero. Debe crear un objeto POSIX de memoria compartida para alojar las dos variables compartidas entre el cajero y los salvajes. (Ayuda: utiliza una estructura que tenga dos campos *last\_servings* y *last\_payment*). Modifica el código de *savage.c* para que éstos tengan acceso a las nuevas variables compartidas y que el número de raciones totales consumidas por cada salvaje sea un número aleatorio entre 3 y 10.
- (1.5 puntos)** Añade a *cashier.c* la creación de los recursos de sincronización necesarios, y completa el código del cajero usando estos recursos para satisfacer las reglas descritas arriba.
- (1.5 puntos)** Modifica el código de *savage.c* para que pueda usar los recursos de sincronización compartidos con *cashier.c*, y añade el código necesario para que, tras haber consumido sus raciones, cada salvaje pague utilizando las reglas descritas arriba.
- (1 punto)** Prepara el programa *cashier.c* para que termine ordenadamente cuando reciba una señal SIGINT o SIGTERM, eliminando los recursos compartidos que haya creado.

**Sugerencia:** en caso de no conseguir que funcione el uso de memoria compartida, apartado a, asume que todos los salvajes comen un número fijo de raciones para hacer los apartados de b a d.

Un ejemplo de ejecución es el siguiente:

|                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>\$ ./savage Savage 17700 obtaining serving from pot Savage 17700 eating Savage 17700 obtaining serving from pot Savage 17700 eating Savage 17700 obtaining serving from pot Savage 17700 eating Savage 17700 paying 30, for 3 servings</pre>                   | <pre>\$ ./savage Savage 17979 obtaining serving from pot Savage 17979 eating Savage 17979 obtaining serving from pot Savage 17979 eating Savage 17979 obtaining serving from pot Savage 17979 eating Savage 17979 paying 30, for 3 servings</pre> |
| <pre>\$ ./cashier The cash register is now open Cashier (pid 17575) collecting 1 payment for 3 serving(s) Total incomes 30 euros for 3 servings served Cashier (pid 17575) collecting 1 payment for 3 serving(s) Total incomes 60 euros for 6 servings served</pre> | <pre>\$ ./cook Cook (pid 17549) putting 10 servings on pot</pre>                                                                                                                                                                                  |